BUSINESS APPLICATIONS OF AI

Module 2 — Value Proposition

Palm Beach Atlantic University · Graduate Program

Value Proposition

> *"A real opportunity is a problem that (1) everyone agrees is a problem, (2) can be solved at scale, and (3) you actually know how to solve. Without all three, you have a hobby."*

Learning Objectives

By the end of this chapter, you will be able to:

- Apply the "pain-first" framework to identify and articulate customer problems with specificity and emotional reson ance.

- Use the Three-Filter Test (Agreement, Scale, Competence) to evaluate and prioritize business opportunities.
- Map customer motivations using Job

s to Be Done—functional, emotional, and social—and apply the "hired and fired" interview technique.

• Distinguish traditional competitive moats from

AI-native moats and evaluate the durability of each in a world where model capabilities commoditize rapidly.

• Analyze the Cursor vs. GitHub Co

pilot case study to identify how workflow depth and positioning, not model access, drove differentiated business outcomes.

• Construct a complete V

alue Proposition Canvas for a real problem domain.

2.1 Pain First, Not Features First

Why 90% of Pitches Die in the First Two Minutes

Investor meetings are predictable. A founder sits down, opens with a slide deck, and within sixty seconds begins describing what their product does: "We built an AI-powered platform that integrates with your CRM to surface predictive insights and automate your outreach workflows." The room goes quiet. Not in anticipation — in polite disconnection.

The mistake is not technical. The pitch may describe a genuinely useful product. The mistake is sequencing: features before pain. When a founder leads with what they built, they are implicitly asking the audience to do the translation work — to imagine what problem this solves, for whom, and whether that problem is worth solving. Most audiences won't do that work. They haven't felt the pain, so they cannot evaluate the cure.

Studies of early-stage investor decision-making consistently find that the ability to articulate the problem clearly — not the sophistication of the solution — is the strongest early predictor of investment interest (Fried & Hisrich, 1994; Maxwell, Jeffrey, & Lévesque, 2011). The investor has heard hundreds of pitches about AI-powered platforms. They have not heard your specific user's pain articulated so precisely that it makes them flinch.

This is the "make them hurt, then heal them" arc — a structure borrowed from classical rhetoric but validated by every effective pitch deck study of the past decade. The sequence is:

```
flowchart LR
    A[Name the Pain] --> B[Make it Visceral]
    B --> C[Quantify the Cost]
    C --> D[Introduce the Cause]
    D --> E[Present the Solution]
    E --> F[Show the Moat]
```

problems we have felt than problems we have only been told about.

Paint the Pain: The Sarah Exercise

The single most powerful tool for a first-time founder is what we call the Sarah Exercise. It is deceptively simple: replace every abstract, category-level pain description with a named individual experiencing a specific moment of acute suffering.

Here is the exercise in action.

Abstract pain description (typical founder language):

> *"Small and mid-size retailers face significant inventory management challenges, leading to inefficiencies and lost revenue opportunities."*

This sentence is technically true of approximately 500,000 businesses in the United States. It is also completely inert. No investor or customer has ever been moved by the phrase "inventory management challenges."

Sarah version:

> *"It's 2:17 in the morning on the Tuesday before Thanksgiving. Sarah Chen, who owns a kitchenware store in Columbus, Ohio, wakes up to find an email from her largest wholesale customer — a regional restaurant chain — ordering $50,000 worth of product for the holiday season. Sarah opens her inventory system and discovers she has 40% of the required stock. She cannot fulfill the order. She emails back to apologize. She loses the order. She loses the customer. She lies awake for the next three hours calculating how she'll make payroll in January."*

The abstract version describes a category. The Sarah version describes a human being at the worst moment in their professional week. The difference in persuasive force is not small — it is categorical.

The Sarah Exercise works because it forces you to answer several questions you should have answered before writing a single line of code:

- Who specifically is experiencin

g this pain? Not a demographic segment — a person with a name, a role, and a bad Tuesday.

- When exactly does the pain occur? Pain has a mo ment. That moment is your product's highest-leverage intervention point.

- What does it cost? Not vaguely — specifically. $50,000 orders, three hours of lost sleep, one customer relationship destroyed.

- What is the emotional consequence? Sarah isn't just experiencing a business setback. She's experiencing fear, shame, and the particular dread of small-business ownership.

Pain Quantification: The Four Dimensions

Once you have your Sarah, you need to measure her pain. Investor conversations and customer discovery interviews require concrete numbers, not impressions. Pain quantification operates across four dimensions:

Dimension	Measure	Example
Time wasted	Hours/week per affected person	Sarah spends 12 hours/week on manual inventory reconciliation
Dollars lost	Revenue lost, costs incurred	$50,000 order lost; $3,400/month in excess carrying costs
Deals missed	Opportunities not captured	2–3 large orders declined per quarter due to stock uncertainty
Sleep lost	A proxy for emotional cost	3 hours of anxiety-sleep after inventory failures

The "sleep lost" dimension is not a joke. It is a shorthand for the emotional weight of a problem — the degree to which it occupies mental space outside work hours. Problems that cost money but don't keep anyone up at night are commodity inconveniences; someone will solve them with a $9/month SaaS tool. Problems that keep people up at night are the kind that justify premium pricing, long-term contracts, and deep customer loyalty.

2.2 The Three-Filter Test

Most aspiring founders have more ideas than discipline. The Three-Filter Test is a structured methodology for killing ideas quickly — before they consume months of your life. Its premise is that a genuine business opportunity must satisfy three conditions simultaneously. Failing any one of them is disqualifying.

Filter 1: Agreement

The question: Do ten unrelated people name the same pain in the same words without being prompted?

This filter is far more demanding than it appears. Many founders conduct "customer discovery" interviews and receive polite agreement: "Yes, that is a real problem." Polite agreement is not the same as unprompted convergence. The agreement filter requires that when you ask ten people in your target market to describe their biggest operational challenge, a significant proportion of them independently reach for the same vocabulary.

When Rob Fitzpatrick, author of The Mom Test (2013), developed his framework for evaluating customer feedback, he observed that most customer discovery is contaminated by what he called the "mom problem" — the tendency of potential customers to tell founders what they want to hear rather than what is true. The Agreement filter bypasses this by looking for convergence, not affirmation.

How to run it:

• Identify 10 potential customers in your target market. Do not pitch your i dea.

- Ask open-ended questions: "What's the hardest part of your week?" "Where do you lose the most time?" "What problem would you pay $1, 000 to solve right now?"

- Record the exact words they use. Do not paraphrase.
- Look for convergence:

do three or more people describe

the same pain using similar language?

If the top answer is scattered — everyone has a different worst problem — the market is not ready for a point solution. If the top answer converges — "we can never get accurate inventory counts" appears in 6 of 10 conversations — you have passed Filter 1.

Filter 2: Scale

The question: Does this problem exist for at least 10,000 addressable entities who could plausibly pay for a solution?

The 10,000 threshold is a practical minimum for a venture-scale business. A problem that affects 200 businesses is a consulting engagement. A problem that affects 10,000 businesses — each paying $2,400/year — is a $24 million ARR business at full penetration, which, even at 10% market share, is a $2.4 million ARR business. That is not a transformative enterprise, but it is a real business.

The scale filter requires a bottom-up market sizing exercise:

```
flowchart TD
    A["Total Addressable Market (TAM)"] --> B["Who has this exact problem?"]
    B --> C["Serviceable Addressable Market (SAM)"] --> D["Who can we reach given our GTM?"]
    D --> E["Serviceable Obtainable Market (SOM)"] --> F["What's realistic in years 1-3?"]
```

Worked example: Suppose you want to build an AI-powered scheduling tool for independent physical therapy practices. Your bottom-up sizing might look like:

- TAM: 39,000 independent physi

cal therapy practices in the US (Bureau of Labor Statistics data)

• SAM: 21,000 practices with 2+ therapists (smaller practices will not pay for software)

- SOM: 2,100 practices (10% market share in years 1–3, given direct sales + referral motion)
- Price point: $299/month = $3,588/year
- SOM revenue:

2,100 × $3,588 = ~$7.5M ARR

$7.5M ARR is not a unicorn. But it is a real business, and if the problem passes Filter 1 (physical therapists consistently naming scheduling as their biggest operational pain), it passes Filter 2.

Filter 3: Competence

The question: Do you have unfair advantages in solving this problem?

The word "unfair" is deliberate. A business advantage that any well-funded team could replicate in six months is not a moat — it is a head start, and head starts erode. Unfair advantages are asymmetric: they derive from who you are, what you know, who you know, or what data you have access to, not just what you built.

Competence can take several forms:

:{tab-item} Domain expertise You have spent years inside the problem domain. You know the vocabulary, the workflows, the regulatory environment, and the specific failure modes. Competitors would need to hire 10 domain experts to replicate your intuition. Example: A former hospital administrator building AI tools for hospital billing has an unfair advantage over a generic fintech team.

Running the Three-Filter Test: A Worked Example

Let us apply all three filters to a candidate idea: an AI assistant for freelance graphic designers to automate client briefing and revision management.

Filter 1 — Agreement: We interview 12 freelance graphic designers and ask: "What's the most frustrating part of client work?" Responses:

- 9/12 mention some version of "uncle

ar briefs" or "constant revision requests"

- 7/12 use the phrase "scope creep"
- 5/12 specifically mention wasting hours responding to emails askin

g for the same information they already provided

Verdict: Passes Filter 1. The pain is converging around unclear briefs, scope creep, and communication overhead.

Filter 2 — Scale:

- Freelance graphic designers in the US: ~330,000 (Bureau of Labor Statistics)
- D

esigners doing primarily client work (vs. in-house): ~180,000

• Designers earning $40,000+/year (can afford software): ~95,000

• Price point: $49/mo

nth = $588/year

- At 5% penetration: 4,750 customers × $588 = ~$2.8M ARR

Verdict: Marginal. $2.8M ARR is a small business, not a venture opportunity. However, the total addressable market expands significantly if the tool is designed for all freelance creative professionals (photographers, copywriters, web designers) — expanding the pool to potentially 500,000+ professionals. With that framing, it passes Filter 2.

Filter 3 — Competence:

- Team member A: Former creative director at a mi d-size agency with 12 years of client-facing creative work

- Team member B: Full-stack developer with UX background
- Domain expertise: Strong (f

ormer CD has lived the problem)

- Proprietary data: None yet, but paid pilot access to 3 agencies for data collection
- Distribution: Team A has LinkedI

n n
et
wo
rk
of
2,4
00
de
sig
ner
s a
nd
cre
ati
ve
dir
ect
ors

Verdict: Passes Filter 3. Domain expertise is genuine, distribution is above average for an early team.

Outcome: This idea passes all three filters. It is not a billion-dollar idea in its current framing, but it is a real opportunity. The team should proceed to customer validation.

2.3 Jobs to Be Done

Christensen's Framework in the AI Era

Clayton Christensen introduced the Jobs to Be Done (JTBD) framework in The Innovator's Solution (2003) and expanded it in Competing Against Luck (2016). The central insight is disarmingly simple: customers don't buy products — they hire outcomes.

When a customer "hires" a product, they are hiring it to make progress on a specific job in their life or work. When they "fire" it, they are firing it because it failed to deliver that progress, or because a better-suited solution became available. The product is a means to an end. The job is the end.

This reframing has profound implications for how you design and pitch a value proposition. A drill manufacturer who understands that customers are "hiring a drill" will compete on drill specifications: RPM, battery life, chuck size. A drill manufacturer who understands that customers are "hiring a hole in the wall" will ask different questions: Why do they need the hole? Could an adhesive hook serve the same job? Could a professional installation service? The JTBD lens constantly pushes the question from "what does our product do?" to "what progress is the customer trying to make?"

In the AI era, the JTBD framework has taken on new relevance. AI tools are generative — they can complete a very wide range of tasks. This creates a temptation to define the job as "use AI to do X." But "use AI to do X" is a feature, not a job. The job is always upstream of the feature. A legal research AI does not exist so lawyers can "use AI for research" — it exists so lawyers can deliver confident, well-researched legal opinions to clients faster and with less anxiety.

Three Dimensions of Jobs

Every customer job has three dimensions:

Functional Jobs The practical, task-level outcome the customer needs to achieve. These are the most visible and the most commonly addressed in product design — but they are also the most easily replicated by competitors.

Example: Sarah needs to know her current inventory levels across all SKUs, updated in real time, accessible from her phone.

Emotional Jobs How the customer wants to feel after hiring your product. These are rarely stated in customer interviews but are consistently present in purchase decisions. Emotional jobs are where differentiation lives — two products with identical functional capabilities will be separated by which one makes the customer feel more capable, more in control, less anxious.

Example: Sarah wants to feel like a competent, organized business owner — not like someone perpetually scrambling to catch up with her own inventory.

Social Jobs How the customer wants to be perceived by others — customers, employees, peers, investors. Social jobs are particularly important in B2B contexts, where purchase decisions often involve organizational signaling as much as functional need.

Example: Sarah wants to be the kind of business owner that restaurant chain buyers trust to deliver reliably. The inventory tool is, in part, a signal of professionalism.

The "Hired and Fired" Interview Technique

The most powerful method for uncovering jobs is the Hired and Fired Interview, a structured qualitative technique developed from Christensen's work and popularized by Bob Moesta and Chris Spiek. The technique focuses on two types of switching events:

- Hired: Tell me about the last time you started using a new tool or service to solve this problem. Walk me through exactly what happened in the days and weeks before you made that decision.

- Fired: Tell

me about the last time you stopped using a tool or service you had been using for this problem. What happened?

The goal is to reconstruct the purchase story and the abandonment story in granular detail. Purchase stories reveal:

• The triggering event (w

hat changed that made the problem urgent enough to act on)

- The struggling moment (the specific instance of pain that motivated the search)
- The

consideration set (what alternatives they evaluated and why they rejected them)

- The hiring criteria (what made them ultimately choose this solution)

Abandonment stories reveal:

- The failure mode (what the product stopped delivering)
- The competing hire (what they switched to and why)
- The unmet emotional job (wh

at they felt the product never really gave them)

Why Customers Don't Buy Products — They Hire Outcomes

The implications of JTBD for value proposition design are direct:

• Your value proposition should name the job, not the product. Instead of "inventory manag ement software for retailers," consider "sleep insurance for small business owners who've been burned by stockouts." The second framing names the emotional job (sleep — safety — control) and the triggering experience (being burned) while implying the functional capability.

- Your competitor set is

broader than your product category. If the job is "feel in control of my business," you are not competing only with other inventory software — you are competing with hiring an operations manager, using spreadsheets obsessively, taking a management course, or simply accepting the anxiety as a cost of ownership. JTBD reveals the full competitive landscape.

• The emotional job determines the price ceiling. Customers will pay more to eliminat e anxiety than to improve efficiency. If your product solves the functional job (saves 12 hours/week), the customer calculates the price of their time and s ets a price ceiling accordingly. If your product solves the emotional job (eliminates the 2am panic), the price ceiling is much higher — because there e is no hourly rate

for peace of mind.

2.4 Moats in an AI-Native World

Traditional Moats and AI-Era Durability

In classical competitive strategy, a moat is a durable structural advantage that allows a company to generate above-market returns for an extended period. Warren Buffett popularized the term; Michael Porter's Five Forces framework provides the analytical scaffolding. Traditional moats include:

Moat Type	Traditional Strength	AI-Era Durability	Why
Brand	Very high	Moderate	AI can replicate brand voice and aesthetic; differentiation requires
Network effects	Very high	High	AI does not dissolve network effects — in fact
Switching costs	High	High	AI-native workflows create *deeper* switching
Scale economies	High	Moderate–Low	AI dramatically reduces marginal cost of
Proprietary IP / patents	Moderate	Low	AI accelerates reverse engineering; patent
Proprietary data	Moderate	Very high	Data is the input to AI; proprietary datasets
Regulatory capture	Moderate	High	Regulatory barriers are AI-agnostic; they persist
Exclusive distribution	Moderate	Moderate	AI improves content production but does not inherently open distribution channels

The table reveals a clear pattern: moats that derive from structural market position (networks, switching costs, regulatory) hold in the AI era. Moats that derive from knowledge production efficiency (scale, IP) erode — because AI democratizes knowledge production.

New AI-Native Moats

The AI era has also created a new class of moat with no direct equivalent in prior competitive strategy frameworks. These AI-native moats are worth understanding in depth because they are poorly recognized by traditional investors and frequently overlooked by founders who focus too heavily on model capability.

Moat 1: Workflow Depth

Workflow depth describes how completely a product has been woven into the daily operational fabric of the customer's work. A tool that an employee opens once a week to generate a report is a peripheral product — replaceable with limited disruption. A tool that an employee interacts with 47 times per day, at every decision point, that has their historical context, their preferences, their team's norms embedded in its outputs — that tool is operationally irreplaceable.

The depth of workflow integration is the primary determinant of switching costs in AI-native products. This is why Notion's AI features are defensible even against more capable standalone AI tools: they are embedded in the document where the work lives. This is why GitHub Copilot, despite its first-mover advantage, was vulnerable to Cursor — as we will explore in the case study below.

Moat 2: Proprietary Feedback Loops

Every interaction a user has with your AI product is a training signal. Every correction, every re-generation, every accepted or rejected suggestion teaches the system something about what good outputs look like for this specific user, company, or domain. Over time, these feedback signals accumulate into a proprietary preference dataset that no competitor can acquire — because it was generated by your users, doing their work, in your product.

This is categorically different from training data that any team can purchase or scrape from the internet. Proprietary feedback loops create personalization depth that compounds over time. A system that has processed 50,000 of Sarah's inventory decisions knows things about how Sarah evaluates tradeoffs that no general model knows — and no competitor can acquire that knowledge without replicating Sarah's history.

Duolingo is a clear example of this moat at scale. Their AI-powered language instruction engine has been trained on hundreds of millions of learning interactions — what spacing intervals work for which types of learners, which error patterns predict future failure, which exercise types produce retention. That dataset cannot be purchased. It can only be generated by having 40 million active learners using the product daily.

Moat 3: Agent Graph Position

As AI systems evolve from single-model tools to multi-agent architectures — networks of specialized AI agents that coordinate to complete complex tasks — position in the agent graph becomes a competitive advantage. An agent that coordinates other agents, routes tasks, and manages state across a workflow holds a structurally superior position to a specialist agent that only performs one function.

This is analogous to platform economics: the entity that controls the coordination layer captures more value than the entities that provide specific capabilities. A company whose AI system sits at the "orchestration layer" of a customer's agent workflow — routing tasks to specialized agents, maintaining memory, managing the human-in-the-loop interface — has a position that is extremely difficult to displace.

This moat is still emerging. As of 2025, most enterprise AI deployments are single-model or loosely coupled tools. The companies that establish orchestration-layer positions in the next 24 months will be extremely difficult to dislodge once multi-agent workflows become standard operating procedure.

Moat 4: Prompt IP and Evals

The final AI-native moat is the least glamorous but arguably the most undervalued: accumulated prompt infrastructure and evaluation capability. This includes:

- Prompt libraries: Curated, tested system prompts that reliably produce high-quality outputs for specific tasks
- Evals: Automated test suites t

hat evaluate whether AI outputs meet quality standards—enabling rapid model upgrades without quality regression

• Fine-tuning datasets: Labeled data

sets derived from human-reviewed outputs that can be used to fine-tune models for specific domains

- Failure mode documentation: Institutional knowl edge of where the model fails, under what conditions, and what guardrails prevent those failures

A team that has spent 18 months building, testing, and refining prompt infrastructure for clinical documentation review has an asset that a new competitor starting from scratch cannot replicate in 6 months — even if they access the same underlying model. The prompt infrastructure represents accumulated knowledge about how to make the model work reliably in a specific high-stakes domain.

Case Study: Cursor vs. GitHub Copilot

In 2023, GitHub Copilot was the dominant AI coding assistant. It had first-mover advantage, the backing of Microsoft and OpenAI, and integration into the world's most widely used code editor (VS Code). It seemed unassailable.

By 2025, Cursor had captured a significant and vocal share of professional developers — particularly in startup environments — and was generating substantial ARR with a fraction of

Copilot's institutional resources. Both products were using similar underlying models. How?

The Setup: Same Models, Different Architectures

GitHub Copilot was designed as a code completion plugin — it operated inside VS Code, suggesting completions as the developer typed. The mental model was "smart autocomplete." The job being hired was narrow: finish my line of code faster.

Cursor was designed as a programmer's AI partner — it was itself an IDE (based on VS Code's open-source core), meaning the AI was not a plugin inside someone else's product but the environment itself. The mental model was "AI-native development." The job being hired was broader: help me build software faster, from architecture through debugging.

The Moat Analysis

Cursor's moats:

- Workflow depth: Because Cursor is the IDE rather than a plugin inside one, it has access to the full project context—every file, every function, every dependency. Copilot, operating as a plugin, had limited context windows that restricted how m uch of the codebase it could "see." Cursor's AI could respond to instructions like "refactor the authentication module to use JWT instead of session cookies across the entire codebase"

— a task requiring full-project visibility that Copilot could not reliably complete.

• Proprietary feedback loops: Ev

ery interaction Cursor users have—every accepted suggestion, every rejection, every manual edit—generates training signal that improves Cursor' s model for subsequent users. Over time, this feedback accumulates into proprietary fine-tuning data for code patterns, error corrections, and develo per preferences.

- Agent graph position: Cursor's "Composer" feature (later expanded as "Agent" mode) positions the AI at the task-orchestration level:

it can create files, run terminal commands, read test outputs, and iterate on code autonomously. This is not a code completion plugin — it is a dev elopment workflow orchestrator. That orchestration position is extremely difficult for a plugin-based competitor to replicate without rebuilding the produ ct from scratch.

- Switching costs: Developers who have used Cursor for 3+ months have their editor preferences, keybindings, and workflow patterns embedded in the tool. The switching cost to return to VS Code + Copilot is non-trivial — not because of data lock-in, but because of workflow ha bit lock-in.

What Copilot missed:

GitHub Copilot's design reflected a conservative product philosophy: do not disrupt the existing workflow, add AI incrementally to what developers already use. This is a sensible risk-minimization strategy that works well in stable markets. In a rapidly evolving AI market, it produced a product that was incrementally better within a workflow architecture that was structurally limiting.

Copilot hired itself as a "typing assistant." Cursor hired itself as a "software development partner." The jobs are not the same, and the products that emerge from those job definitions are not the same.

What the Moat Was

The Cursor moat was not a better AI model. It was:

- Full-project context (technical architecture choice enabling workflow depth)
- Orchestration-lay

er positioning (agent graph position)

- Accumulated developer feedback (proprietary feedback loops)
- Workflow habit formation (switching costs)

This is the template for AI-native competitive advantage. The companies that win in the AI era will not win because they have access to better models — models are increasingly available to everyone. They will win because they have built systems that accumulate proprietary advantage the longer customers use them.

Faith Integration — Module 2

> *"Whatever you do, work at it with all your heart, as working for the Lord, not for human masters."*
> *— Colossians 3:23 (NIV)*

Excellence as Worship: The Theology of a Great Value Proposition

Colossians 3:23 is one of the most practically disruptive verses in the New Testament. It doesn't say "do your best when it matters." It says whatever you do — including, implicitly, the spreadsheets, the pitch decks, the customer interviews, and the product specs — do it wholeheartedly, as if the audience is God himself.

This verse reframes what it means to build a great value proposition. A pain-first value proposition is not merely good strategy; it is an act of attention — of actually listening to another person's struggle rather than projecting your solution onto them. The Jobs-to-Be-Done framework is, at its core, an exercise in empathy. What is this person really trying to accomplish? What is getting in their way? What would it mean for their life if that problem disappeared?

Christian business professionals have a theological basis for this kind of deep customer attention that secular frameworks cannot fully supply. We are commanded to love our neighbors as ourselves (Matthew 22:39). Your customer is your neighbor. Building a value proposition that genuinely solves their problem — not one that merely generates revenue — is an act of love expressed through commerce.

The standard Colossians sets is demanding precisely because it refuses to let us compartmentalize: our professional craft is not separate from our faith. The quality of care we put into our value proposition is a form of worship.

' þ Faith Integration Paper — Module 2

Assignment: Using Claude or Gemini as a co-writer, compose a 1-page reflection paper (approximately 300–400 words) responding to the following prompt:

> *Colossians 3:23 calls you to work wholeheartedly "as for the Lord." Apply this to the value proposition you are developing in this course. What would it look like to build a product or service for your target customer with that same standard of care? Where are you tempted to cut corners on genuinely understanding their pain? What would change if you treated customer discovery as a spiritual discipline?*

Process:

• S
har

e the prompt with Claude or Gemini and use the AI to brainstorm and draft.

- Revise into your own voice — include a real example from your custo mer research or life experience.

• Submit the final, human-owned version.

Format: 1 page, double-spaced, 12pt font, Times New Roman. Include Colossians 3:23 at the top.

Grading: Theological engagement, personal authenticity, connection to your actual value proposition work.

Lab 2: The Value Proposition Canvas

Estimated time: 3–4 hours

Overview: The Value Proposition Canvas (VPC) is a tool developed by Alexander Osterwalder and Yves Pigneur that maps customer jobs, pains, and gains against a product's value map. It is the most widely used structured framework for designing and evaluating value propositions in the startup ecosystem.

In this lab, you will complete a VPC for a real problem domain, write a one-paragraph value proposition, and conduct a preliminary moat analysis.

Deliverable: (1) A completed Value Proposition Canvas (template or hand-drawn, photographed), (2) a one-paragraph value proposition statement, and (3) a moat analysis identifying which of the four AI-native moats your proposed solution could develop.

Lab Tasks

Task 1: The Sarah Exercise — Persona and Pain Narrative (45 min)

- Select a business domain you know well — your industry, a prior employer's industry, or a market you have researched.
- Identify the

single biggest operational pain in that domain. Write it as a category-level description first (one sentence).

- Transform it using the Sarah Exercise: write a

150–200 word narrative naming a specific person, at a specific moment, experiencing acute pain. Include at least one quantified cost.

- Use the following

starter prompt with your AI tool of choice (after meta-prompting it): "Help me transform this category-level pain description into a vivid, specific pers on a narrative: [paste your description]. The narrative should name a specific person, describe a specific bad moment, include a quantified cost, and surf ace the emotional consequence."

Task 2: Customer Discovery — The Three-Filter Test (60 min)

- Take three candidate business ideas from your own experience, research, or interest.
- For each ide

a, complete the Three-Filter Test:

- Filter 1 (Agreement): What evidence do you have that

10 people name this pain unprompted? If you lack evidence,

design a 5-question interview protocol to gather it.
- Filter 2 (Scale): Conduct a bottom-up market sizing using available public data. Show your math.
- Fi

lter 3 (Competence): List your team's unfair advantages. Be honest about gaps.

- Use this starter prompt (after meta-prompting): "I'm evaluating a busine ss idea using the Three-Filter Test. Here is the idea: [description]. Help me assess whether it passes each of the three filters: Agreement (do customers in dependently name this pain?), Scale (does the market support a real business?), and Competence (does my team have unfair advantages?). Identify the weakest filter and suggest how to address it."

Task 3: Jobs to Be Done Interview (60 min)

• Identify one person who has recently purchased or stopped using a product in your target do main.

- Conduct a 30-minute hired or fired interview using the script in Section 2.3.
- After the interview, write a 200-word synthesis that answers

:
(a) What was the triggering event?
(b) What was the functional job?
(c) What was the emotional job?
(d) What was the social job?

- Use this starter

prompt (after meta-prompting): "I conducted a customer interview and here are my notes: [paste notes]. Help me identify the functional job, emotional job, and social job the customer was trying to accomplish. What does this suggest about how I should frame the value proposition?"

Task 4: Value Proposition Canvas (60 min)

- Download the V

PC template from Strategyzer or recreate it.

- Complete the Customer Profile side: list 5+ customer jobs, 5+ customer pains (ranked by intensity), and

3+ customer gains.

• Complete the Value Map side: list your proposed product/service, identify which pains it relieves and how, and identify which gains s it creates.

• Achieve fit: circle the pains and gains on the Customer Profile that your Value Map directly addresses. A strong VPC addresses at least 3 of the top-5 pains.

- Use this starter prompt (after meta-prompting): "Here is my completed Value Proposition Canvas: [describe both sides]. Evaluate th e strength of the fit between my value map and customer profile. Identify the three biggest gaps. Suggest how I could redesign the value proposition to achieve stronger

fit."

Task 5: Moat Analysis (45 min)

• Review the four

AI-native moats from Section 2.4: Workflow Depth, Proprietary Feedback Loops, Agent Graph Positio n, and Prompt IP.

• For your proposed solution, write one paragraph per moat: (a) Does this moat apply to your solution? (b) How would you dev elop it? (c) What is the timeline to build meaningful depth?

• Use this starter prompt (after meta-prompting): "I'm analyzing the competitive moats for thi s AI-native product: [description]. Evaluate which of the four AI-native moats (workflow depth, proprietary feedback loops, agent graph position, promp t IP and evals) apply to this product. For the two strongest, describe specifically how the company should invest to build those moats over the next

18 months."

AI Studio Build — Weekly Application

The Pain Translator

Every business problem starts as an abstraction. "Supply chain inefficiency." "Customer churn." "Employee engagement." These abstractions are analytically inert — they tell you nothing about who is suffering, when, or how much. The Pain Translator is an AI Studio application that converts abstract problem statements into structured, actionable pain narratives.

What you are building: In aistudio.google.com, build a structured-output prompt that takes an abstract problem statement (e.g., "inventory challenges for small retailers") and returns a JSON object containing:

- persona: a named, specific person with a role, company type, and location — not a de mographic category

- pain_moment: the specific moment of acute pain — include the time, place, what happened, and who was affected
- quantif

ied_cost: a concrete measure of the cost—time lost in hours, dollars lost, or deals missed

- emotional_consequence: how the persona felt in that m oment and in the hours after — specific emotional language, not generic descriptors

- root_cause: the structural reason why this problem occurred —

what organizational, technical, or market failure created the conditions

Step-by-step:

- Open aistudio.google.com and navigate to Build.
- Select a Gemini

model and enable JSON mode (or "Structured output" — the toggle is in the model settings panel).

- Define a response schema that enforces the fiv e-field structure above. In AI Studio, this is done by clicking "Edit Schema" and defining the JSON schema:

```
    "pain_moment": { "type": "string" },
    "quantified_cost": { "type": "string" },
    "emotional_consequence": { "type": "string" },
    "root_cause": { "type": "string" }
  },
  "required": ["persona", "pain_moment", "quantified_cost", "emotional_consequence",
"root_cause"]
}
```

te a system

instruction that tells the model: (a) its role as a business pain analyst, (b) the level of specificity required (named person, specific moment, quantified c ost), and (c) the quality bar (outputs must be visceral and concrete, not generic or abstract). Apply meta-prompting to develop a thorough 2–3 paragraph system instruction.

- Test your Pain Translator with at least three different abstract problem statements from three different industries:
  - A retail operations p

roblem
- A healthcare operations problem
- A financial services or professional services problem

• For each test, evaluate the output: Is the persona specific and believable? Is the pain moment vivid? Is the cost quantified? Refine your system instruction based on what you observe.

• Use the Share fu

nction to generate a public AI Studio share link.

Deliverable: Submit (a) your AI Studio share link, (b) the full JSON schema you defined, and (c) three sample Pain Translator outputs for three different problem statements. Your system instruction should be 2–3 substantial paragraphs.

What you are learning: Structured output, JSON mode, response schema definition. This is the foundational capability for any AI application that needs to produce machine-readable output rather than freeform text — essential for every subsequent AI Studio Build assignment and for any production AI pipeline you will build.

Discussion Prompts

Discussion 1: The Pain-First Inversion in Your Industry

Select a product or service in your industry that you believe was built "features first" — designed around a technical capability before a validated customer pain. Using the frameworks from this chapter, reconstruct what a "pain-first" version of that product's development might have looked like.

In your post:

- Identify the specific feature or capabilit y the company led with, and what customer pain you believe it should have led with

• Describe what the "Sarah version" of that pain would look like

— a named persona, a specific bad moment, a quantified cost

- Apply the Three-Filter Test: does the pain you identified pass all three filters?
- A

rgue whether the company's features-first approach was a strategic error or a pragmatic product decision given market conditions at the time

Support your argument with at least two scholarly sources on value proposition design, customer development, or innovation strategy. Include at least one current source (no more than two years old) documenting outcomes for the company you selected.

Discussion 2: The Moat That Isn't a Moat

Many AI companies currently claim competitive advantages that are, under careful analysis, not durable moats — they are first-mover advantages that will erode as model capabilities improve and competitive intensity increases.

In your post:

- Select a real AI company (public or well-documented private) and identify the competitive advantage they most promine ntly claim

- Apply the Moat Durability Matrix from Section 2.4: is this a durable moat or an eroding one?
- Identify which of the four AI-native moats (wo

rkflow depth, feedback loops, agent graph position, prompt IP) the company is most likely to develop over the next three years—or argue that they will no t develop a durable moat at all

• Make a prediction: will this company still be among the top three players in its category in three years? Defend your pr ediction with evidence.

Your post should demonstrate familiarity with at least one of the following scholarly frameworks: Porter's Five Forces (Porter, 1980), competitive dynamics (D'Aveni, 1994), or resource-based view of the firm (Barney, 1991).

Readings

Required:

- Osterwalder, A., Pigneur, Y., Bernarda, G., & Smith, A. (2014). Value proposition design: How to

create products and services customers want. Wiley. (Chapters 1–2: The Value Proposition Canvas)

• Christensen, C. M., Hall, T., Dillon, K., & Dunc an, D. S. (2016). Know your customers' "jobs to be done." Harvard Business Review, 94(9), 54–62. Available at hbr.org

- Fitzpatrick, R. (2013). The Mom Te

st: How to talk to customers and learn if your business is a good idea when everyone is lying to you. Robfitz Ltd. (Chapters 1–3)

• Porter, M. E. (

1980). Competitive strategy: Techniques for analyzing industries and competitors. Free Press. (Chapter 1: The Structural Analysis of Industries)

Recommended:

- Christensen, C. M., & Raynor, M. E. (2003). The innovator's solution: Creating and sustaining successful growth. Harvard Business Review Press.
- Moe

sta, B. (2023). Demand-side sales 101: Stop selling and help your customers make progress. Lioncrest Publishing.

- Barney, J. (1991). Firm resources and su stained competitive advantage. Journal of Management, 17(1), 99–120.

- D'Aveni, R.A. (1994). Hypercompetition: Managing the dynamics of strategic m aneuvering. Free Press.

- Blank, S., & Dorf, B. (2012). The startup owner's manual: The step-by-step guide for building a great company. K&S Ranc

h P
res
s.

Glossary

Term	Definition
Value Proposition	The specific combination of benefits a product or service delivers to a defined customer segment, addressing their jobs, pains, and gains in a way that
Pain-First Framework	A product development and pitch methodology that prioritizes identifying and articulating customer pain
The Sarah Exercise	A technique for transforming abstract, category-level pain descriptions into specific, vivid persona narratives that name a real person, a real moment, and a real cost
Pain Quantification	The practice of measuring customer pain across four dimensions: time wasted, dollars lost, deals missed
Three-Filter Test	A structured framework for evaluating business opportunities across three criteria: Agreement (do customers independently name the same pain?),
Agreement Filter	The first filter in the Three-Filter Test: a problem passes when 10 unrelated people name the same pain
Scale Filter	The second filter in the Three-Filter Test: a problem passes when it affects at least 10,000 addressable entities who could plausibly pay for a solution
Competence Filter	The third filter in the Three-Filter Test: a problem passes when the team has unfair, asymmetric ... data, distribution)
Jobs to Be Done (JTBD)	Clayton Christensen's framework holding that customers do not buy products — they hire solutions to make progress on specific jobs in their lives or work
Functional Job	The practical, task-level outcome a customer needs to achieve; the most visible dimension of JTBD
Emotional Job	How the customer wants to feel after hiring a product; the motivating dimension of JTBD that determines willingness to pay premium prices
Social Job	How the customer wants to be perceived by others after using a product; particularly important in B2B
Hired and Fired Interview	A structured qualitative technique that reconstructs purchase stories (hired) and abandonment stories (fired) to uncover the true jobs customers are trying to accomplish
Competitive Moat	A durable structural advantage that allows a company to generate above-market returns for an extended
Workflow Depth	An AI-native moat describing how deeply an AI product is embedded in the customer's daily operational fabric, creating switching costs through habit and integration
Proprietary Feedback Loop	An AI-native moat in which user interactions generate training signals that improve the AI system, creating a
Agent Graph Position	An AI-native moat derived from occupying the orchestration or coordination layer in a multi-agent AI system, capturing value from routing, memory
Prompt IP	An AI-native moat consisting of accumulated prompt libraries, evaluation frameworks, fine-tuning datasets
Value Proposition Canvas (VPC)	A strategic design tool by Osterwalder and Pigneur that maps customer jobs, pains, and gains against a product's value map (products/services, pain relievers, gain creators) to achieve value proposition fit
Capability Commoditization	The process by which AI model capabilities that once represented competitive advantages become widely available, eroding moats built on model access rather than structural position